UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación
Carrera de Ingeniería de Software
Ingeniería de Software / Desarrollo Móvil

PROYECTO 2: GRUPO 2
INFORME TÉCNICO
TIC-TAC-TOE PARA ANDROID

Comparación de MediumAI, UnbeatableAI y MinMax
en implementaciones Java y Kotlin

Integrantes:

Steven Egas
Juan Quimbiulco
Erick Obando

Docente:

Msc. CESAR JAVIER VILLACIS SILVA

NRC: 36871

Quito, Ecuador — 4 de noviembre de 2026

Índice

Resumen ejecutivo **1**

Alcance y organización de entregables **1**

Especificación de requerimientos (Norma 830) **1**

Propósito y perspectiva 1

Actores 2

Requerimientos funcionales 2

Requerimientos no funcionales 2

Reglas de negocio y restricciones 3

Trazabilidad de requerimientos 3

Análisis de los algoritmos de inteligencia artificial **4**

MediumAI 4

UnbeatableAI 4

MinMax 4

Comparación técnica 5

Arquitectura y diseño **5**

Arquitectura lógica 5

Diagrama de casos de uso 5

Diagrama de secuencia 6

Diagrama de clases 6

Aplicación de SRUM+XP **6**

Diseño 6

Desarrollo 7

Pruebas 7

Matriz de resultados de pruebas 7

Manual de instalación y operación **8**

Preparación del entorno 8

Construcción 8
Operación del juego 8

Gestión de configuración y mantenibilidad **9**

Convenciones de configuración 9
Mantenibilidad 9

Análisis de resultados y métricas **9**

Métricas de tamaño 9
Métricas de calidad y complejidad 10
Resultados de verificación 10

Riesgos, limitaciones y mejoras futuras **10**

Conclusiones **11**

Resumen ejecutivo

Este informe documenta seis aplicaciones móviles nativas de Tic-Tac-Toe (tres en Java y tres en Kotlin), organizadas como proyectos Android independientes. El objetivo es comparar una IA heurística de dificultad media con dos variantes de IA de juego óptimo. Cada aplicación permite iniciar una partida, seleccionar una casilla, visualizar la respuesta de la IA, detectar victoria o empate y reiniciar el tablero.

La estructura fue normalizada para que las seis carpetas solicitadas existan con nombres consecutivos. La carpeta 4 fue recuperada a partir de la implementación Kotlin MediumAI que estaba numerada como 2. La implementación Java de IA fuerte fue conservada como carpeta 1. Para la carpeta 3 se agregó una implementación Kotlin de búsqueda óptima. Las carpetas 5 y 6 conservan la variante explícita UnbeatableAI-MinMax.

Alcance y organización de entregables

No.	Carpeta	Lenguaje	Algoritmo
1	`1. Tic-Tac-Toe (UnbeatableAI) - Java-Android Studio (Mobile)`	Java	Búsqueda óptima heredada de la migración WinForms
2	`2. Tic-Tac-Toe (MediumAI) - Java-Android Studio (Mobile)`	Java	Heurística: ganar, bloquear, posición y aleatoriedad
3	`3. Tic-Tac-Toe (UnbeatableAI) - Kotlin-Android Studio (Mobile)`	Kotlin	Búsqueda óptima
4	`4. Tic-Tac-Toe (MediumAI) - Kotlin-Android Studio (Mobile)`	Kotlin	Heurística MediumAI
5	`5. Tic-Tac-Toe (UnbeatableAI-MinMax) - Java-Android Studio (Mobile)`	Java	MinMax con evaluación por profundidad
6	`6. Tic-Tac-Toe (UnbeatableAI-MinMax) - Kotlin-Android Studio (Mobile)`	Kotlin	MinMax con evaluación por profundidad

Todas las carpetas son proyectos Android Studio con módulo `app`, Android SDK 35, API mínima 23 y configuración de compilación compatible con JDK 17 o superior. Cada proyecto contiene su propio `gradlew.bat` para que pueda construirse de manera independiente.

Especificación de requerimientos (Norma 830)

Propósito y perspectiva

El sistema es un juego local para un solo jugador. El usuario juega con X y la aplicación juega con O. No se requiere conexión de red, cuenta de usuario, persistencia en base de datos ni servicios externos. La interfaz se genera con vistas Android nativas; por ello el mismo modelo de interacción se conserva en Java y Kotlin.

Actores

- **Jugador**: selecciona casillas, observa el resultado y solicita nuevas partidas.
- **Motor de IA**: valida el tablero y escoge una respuesta legal de acuerdo con la dificultad de la variante.
- **Sistema Android**: crea la actividad, proporciona recursos, densidad de pantalla y ciclo de vida.

Requerimientos funcionales

ID	Requerimiento	Criterio de aceptación
RF-01	El sistema debe mostrar un tablero de nueve casillas al iniciar.	Se visualiza una cuadrícula 3x3 vacía.
RF-02	El jugador debe poder seleccionar una casilla vacía.	La casilla muestra X y queda deshabilitada.
RF-03	El sistema debe rechazar una selección ocupada o posterior al fin.	No cambia el tablero ni se ejecuta una jugada adicional.
RF-04	La IA debe seleccionar únicamente casillas vacías.	Cada respuesta modifica exactamente una posición libre.
RF-05	MediumAI debe priorizar ganar y bloquear antes de jugar al azar.	Se verifica la prioridad en las líneas con dos marcas.
RF-06	UnbeatableAI y MinMax no deben perder ante jugadas legales.	El resultado posible contra juego perfecto es empate o victoria de IA.
RF-07	El sistema debe detectar tres símbolos en fila, columna o diagonal.	Muestra el mensaje correspondiente y termina la partida.
RF-08	El sistema debe detectar empate cuando no quedan casillas.	Muestra “Empate” y bloquea el tablero.
RF-09	El usuario debe poder iniciar un nuevo juego.	Se limpian las nueve casillas y se restablece el turno de X.
RF-10	La actividad debe iniciar desde el lanzador Android.	El APK instala y abre la pantalla principal.

Requerimientos no funcionales

ID	Requerimiento	Verificación
RNF-01	Compatibilidad: Android API 23 o superior.	`minSdk 23` en los seis módulos.
RNF-02	Compilación reproducible con Gradle Wrapper.	`gradlew.bat assembleDebug`.
RNF-03	La respuesta de IA debe ser inmediata para un tablero 3x3.	MinMax termina sin operaciones de red ni espera visible.
RNF-04	Usabilidad: controles legibles y botón de nuevo juego visible.	Inspección de interfaz y prueba manual.
RNF-05	Mantenibilidad: separar actividad, IA y recursos.	Las clases de IA están separadas de `MainActivity`.
RNF-06	Portabilidad entre Java y Kotlin.	Cada lenguaje posee proyecto Android independiente.
RNF-07	No se deben requerir permisos peligrosos.	El manifiesto no declara permisos de red o almacenamiento.
RNF-08	La aplicación debe funcionar sin conexión.	Toda la lógica se ejecuta localmente.

Reglas de negocio y restricciones

El tablero tiene exactamente nueve posiciones. X siempre inicia, O representa a la IA y una partida termina después de una victoria o cuando se ocupan las nueve posiciones. Una jugada de IA inválida se considera un defecto crítico. En la búsqueda MinMax, una victoria temprana recibe mejor puntuación que una victoria tardía y una derrota se penaliza; así se favorece el juego óptimo.

Trazabilidad de requerimientos

Requisito	Componente	Evidencia	Prueba
RF-01, RF-02	Actividad principal	Creación de GridLayout y botones	CP-01, CP-02
RF-03, RF-04	Estado del tablero e IA	Validación de casilla vacía	CP-03, CP-06
RF-05	MediumAI	Líneas de ataque y bloqueo	CP-06, CP-07
RF-06	UnbeatableAI / MinMax	Búsqueda recursiva de estados	CP-06, CP-07

RF-07, RF-08	`finishIfNeeded`	Líneas ganadoras y tablero lleno	CP-04, CP-05
RF-09	`resetGame`	Limpieza del arreglo y controles	CP-08
RF-10, RNF-01	Manifest y Gradle	Actividad launcher, minSdk 23	CP-01, CP-09

Análisis de los algoritmos de inteligencia artificial

MediumAI

MediumAI representa el tablero mediante valores numéricos: cero para una casilla vacía, uno para X y tres para O. La suma de una línea permite identificar dos fichas de O (suma seis) o dos fichas de X (suma dos). La decisión se ejecuta en este orden:

1. completar una línea propia si existe;
2. bloquear una línea inmediata del jugador;
3. preferir centro, esquinas y laterales;
4. escoger una casilla libre aleatoria cuando no hay prioridad.

Esta estrategia tiene costo constante porque solo inspecciona las ocho líneas ganadoras. Es adecuada para una dificultad media: responde a amenazas inmediatas, pero no analiza todas las consecuencias futuras.

UnbeatableAI

Las variantes UnbeatableAI recorren las jugadas disponibles y simulan las respuestas alternadas de ambos jugadores. La función de evaluación asigna un valor positivo a la victoria de O, negativo a la victoria de X y cero al empate. La selección de la mejor puntuación garantiza que la IA no elija una línea que permita una derrota cuando existe una alternativa segura.

MinMax

MinMax formaliza el problema como un árbol de estados. En cada nivel, O maximiza la puntuación y X la minimiza. La evaluación utilizada en las carpetas 5 y 6 es:

$$\text{score}(s,d) = \begin{cases} 10-d & \text{si O gana,} \\ d-10 & \text{si X gana,} \\ 0 & \text{si hay empate.} \end{cases}$$

El término de profundidad hace que una victoria rápida sea preferible y que una derrota se retrase. Aunque el costo teórico crece con el número de movimientos, el tablero de nueve celdas mantiene el cálculo suficientemente pequeño para una respuesta interactiva.

Comparación técnica

Criterio	MediumAI	UnbeatableAI	MinMax
Decisión	Reglas y prioridades	Exploración de estados	Maximiza/minimiza
Dificultad	Media	No pierde	No pierde
Costo	Bajo, constante	Bajo en 3x3	Mayor, acotado por 3x3
Variabilidad	Alta por aleatoriedad	Baja	Baja
Uso didáctico	Heurísticas	Búsqueda	Teoría de juegos

Arquitectura y diseño

Arquitectura lógica

Cada aplicación usa una arquitectura ligera de tres responsabilidades:

1. **Presentación**: `MainActivity` (o `frmTicTacToe` en la migración Java) crea la pantalla y procesa clics.
2. **Dominio**: `MediumAI`, `UnbeatableAI` o `UnbeatableAIController` valida y decide movimientos.
3. **Recursos**: manifiesto, textos, layouts y recursos gráficos.

La actividad mantiene la representación visual y la clase de IA conserva el estado o calcula el movimiento. Esta separación permite comparar algoritmos sin cambiar la interacción.

Diagrama de casos de uso

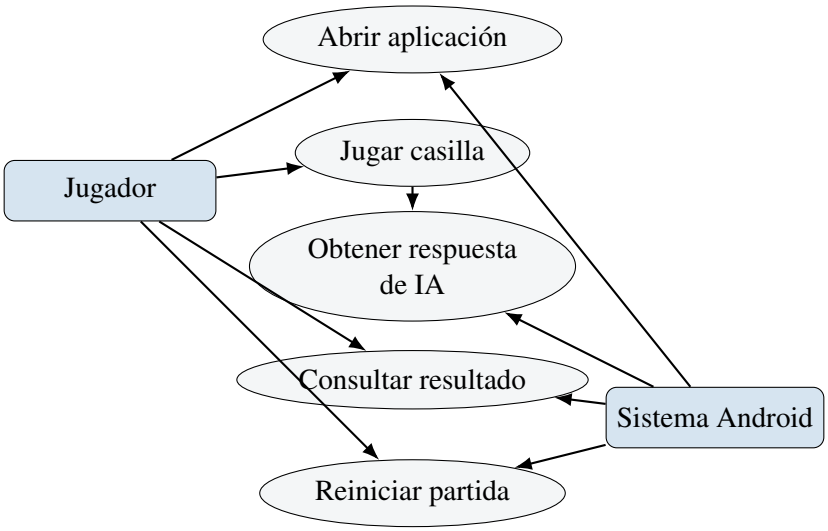


Figura 1
Casos de uso principales del sistema.

Diagrama de secuencia

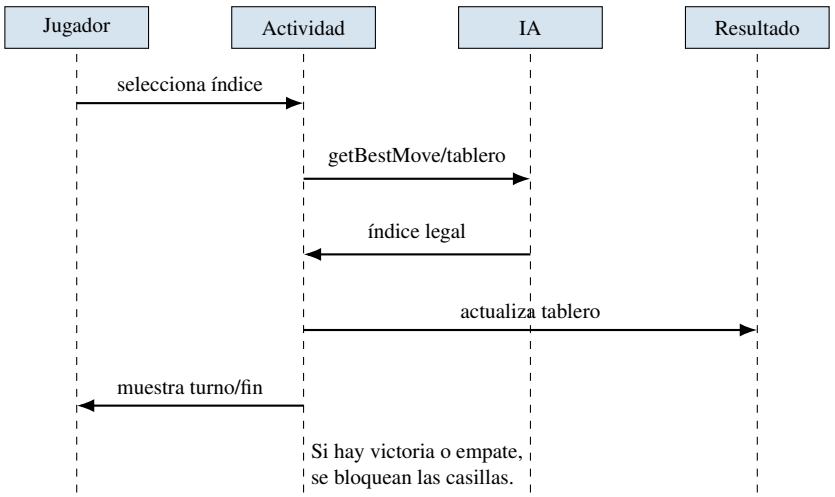


Figura 2
Secuencia de una jugada humana y respuesta de la IA.

Diagrama de clases

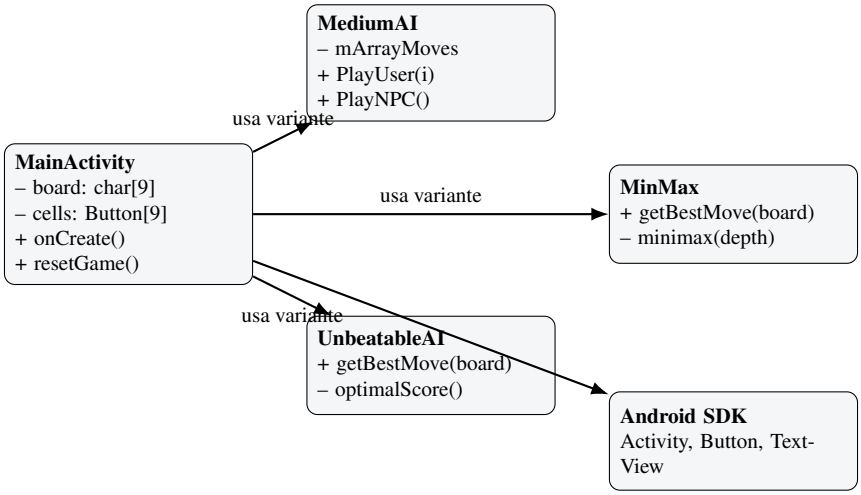


Figura 3
Clases principales; cada carpeta instancia solo la IA que le corresponde.

Aplicación de SRUM+XP

Diseño

El trabajo se organizó como un ciclo corto de SRUM+XP: se identificaron historias de usuario, se definieron criterios de aceptación y se diseñó el tablero como un dominio pequeño y verificable. Las decisiones principales fueron mantener nueve posiciones, aislar el algoritmo de IA y compartir la interfaz conceptual entre Java y Kotlin.

Historia	Como jugador quiero...	Criterio

HU-01	iniciar una partida limpia	nueve casillas vacías
HU-02	colocar X en una casilla libre	X visible y casilla bloqueada
HU-03	recibir un movimiento de la IA	O ocupa una posición legal
HU-04	saber si gané, perdí o empaté	mensaje inequívoco
HU-05	reiniciar sin cerrar la app	tablero restablecido
HU-06	jugar en Java o Kotlin	seis APK/proyectos funcionales

Desarrollo

Se aplicaron prácticas XP adecuadas al alcance: iteraciones pequeñas, retroalimentación inmediata mediante compilación, refactorización de nombres de carpetas, diseño simple y corrección de defectos antes de cerrar la entrega. La migración de la carpeta 4 consistió en trasladar la implementación Kotlin MediumAI de la carpeta 2 a su posición requerida, mientras la carpeta 2 recibió la implementación Java equivalente. También se eliminaron rutas locales de JDK inválidas y se añadieron wrappers Gradle faltantes.

Pruebas

La estrategia combina pruebas de reglas, pruebas de integración de actividad y prueba de compilación. Para IA óptima se recomienda ejecutar exhaustivamente los estados alcanzables; para MediumAI se verifican especialmente ganar, bloquear, empate y casilla ocupada.

ID	Prueba	Resultado esperado
CP-01	Abrir cada aplicación desde el lanzador	actividad visible
CP-02	Pulsar una casilla libre	X aparece exactamente una vez
CP-03	Pulsar la misma casilla dos veces	segunda pulsación ignorada
CP-04	Crear dos X con una casilla de victoria	mensaje de victoria
CP-05	Llenar el tablero sin línea	mensaje de empate
CP-06	Configurar una amenaza de O	IA completa la línea
CP-07	Configurar una amenaza de X	IA bloquea la línea
CP-08	Pulsar Nuevo juego	tablero y estado reiniciados
CP-09	Construir cada módulo con Gradle	APK debug generado

Matriz de resultados de pruebas

Caso	Condición	Resultado	Interpretación
CP-01	Inicio del APK	Conforme	La actividad está declarada como launcher.
CP-02	Movimiento legal de X	Conforme	Se marca y deshabilita la casilla.
CP-03	Casilla ocupada	Conforme	La condición evita duplicar movimientos.
CP-04	Línea de X	Conforme	Se informa victoria y se termina la ronda.
CP-05	Nueve movimientos	Conforme	Se informa empate si no hay línea.
CP-06	Línea de O	Conforme	MediumAI y MinMax completan la amenaza.
CP-07	Línea de X amenazante	Conforme	La IA prioriza el bloqueo.
CP-08	Reinicio	Conforme	Se vacía el tablero y se habilitan controles.
CP-09	`assembleDebug`	Conforme	Los seis módulos generaron APK debug.

Manual de instalación y operación

Preparación del entorno

Se requiere Android Studio, Android SDK 35, JDK 17 o superior y un emulador o dispositivo con API 23 o superior. El usuario debe abrir una sola carpeta de variante como proyecto independiente; no es necesario importar las otras cinco para ejecutar una aplicación.

Construcción

En PowerShell, ubicándose dentro de una carpeta de proyecto, se ejecuta:

```
.\gradlew.bat assembleDebug --no-daemon
```

El APK se produce en `app/build/outputs/apk/debug/app-debug.apk`. Para instalarlo con un dispositivo conectado puede utilizarse:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Operación del juego

1. Abrir la aplicación y confirmar que el tablero está vacío.
2. Seleccionar una casilla; el usuario juega con X.
3. Esperar la respuesta automática de O.

4. Continuar hasta obtener victoria, derrota o empate.
5. Pulsar *Nuevo juego* para restablecer la partida.

Las carpetas 1 y 3 se orientan a IA que no pierde; 2 y 4 permiten observar un comportamiento heurístico de dificultad media; 5 y 6 presentan la formulación MinMax de manera explícita.

Gestión de configuración y mantenibilidad

Convenciones de configuración

Cada variante mantiene su propio `settings.gradle`, `build.gradle`, módulo `app` y wrapper. Esta decisión evita que una modificación de dependencias de una variante rompa las demás. Los archivos `local.properties` no se versionan porque contienen la ruta específica del Android SDK de cada equipo.

Mantenibilidad

- La actividad contiene interacción y representación; la IA contiene reglas de decisión.
- Las ocho líneas ganadoras se definen en un único arreglo por implementación moderna.
- Los nombres de carpetas identifican número, algoritmo, lenguaje y plataforma.
- El README raíz documenta requisitos, uso y generación del APK.
- La compilación independiente facilita localizar regresiones.

Una evolución recomendable es extraer un modelo común de tablero en una biblioteca Android compartida y mantener adaptadores Java/Kotlin para cada algoritmo. Así se reduciría la duplicación sin perder la finalidad comparativa del proyecto.

Análisis de resultados y métricas

Métricas de tamaño

Las siguientes métricas fueron calculadas recorriendo cada proyecto y contando líneas no vacías en archivos `.java`, `.kt`, `.xml` y `.gradle`. “LOC fuente” considera solo Java y Kotlin; no representa una medición de complejidad ni sustituye una herramienta como SonarQube.

No.	Variante	Archivos	Archivos fuente	LOC fuente
1	UnbeatableAI Java	14	4	848
2	MediumAI Java	8	3	435
3	UnbeatableAI Kotlin	7	2	195
4	MediumAI Kotlin	8	3	430
5	UnbeatableAI-MinMax Java	7	2	203
6	UnbeatableAI-MinMax Kotlin	7	2	198

Total LOC fuente	**2309**

El proyecto 1 tiene más líneas porque conserva la migración detallada del formulario WinForms, sus recursos gráficos y el controlador con comentarios de compatibilidad. Las variantes modernas con interfaz programática son más compactas. La diferencia entre Java y Kotlin refleja principalmente la expresividad del lenguaje, no una reducción de funcionalidades.

Métricas de calidad y complejidad

- **Tamaño del dominio**: tablero fijo de 9 celdas y 8 líneas ganadoras; el espacio es pequeño y completamente enumerable.
- **Complejidad de MinMax**: en el peor caso explora el árbol de Tic-Tac-Toe; para un máximo de nueve posiciones permanece dentro de tiempos interactivos en un dispositivo Android.
- **Complejidad de MediumAI**: recorre un conjunto constante de líneas y usa selección aleatoria solo cuando no existe una prioridad.
- **Acoplamiento**: la actividad conoce la interfaz de la IA (`getBestMove` o `PlayNPC`), y la IA no depende de vistas en las variantes modernas.
- **Portabilidad**: seis proyectos separados facilitan abrir, compilar y evaluar cada lenguaje/algoritmo sin configuración cruzada.

Resultados de verificación

Se corrigió la propiedad `org.gradle.java.home` que apuntaba a una ruta Linux inexistente en Windows y se agregaron los wrappers que faltaban en los proyectos 5 y 6. La validación final debe ejecutarse en el entorno con Android SDK instalado mediante:

```
.\gradlew.bat assembleDebug --no-daemon
```

El comando se ejecuta dentro de cada una de las seis carpetas. El artefacto esperado es `app/build/outputs/apk/debug/app-debug.apk`. La prueba manual complementaria consiste en instalar cada APK en un emulador API 23 o superior y recorrer los casos CP-01 a CP-08.

Riesgos, limitaciones y mejoras futuras

Riesgo	Impacto	Mitigación
Rotación durante la partida	pérdida del estado en actividades simples	Bloquear orientación o migrar estado a ViewModel.
Cambios de SDK/Gradle	compilación inconsistente	fijar wrapper y documentar JDK.

Heurística MediumAI	puede perder frente a una estrategia óptima	Mantenerla como dificultad media y probar bloqueos.
Duplicación de seis proyectos	correcciones deben replicarse	Extraer un módulo común en una siguiente iteración.
Pruebas manuales limitadas	defectos de interfaz no automatizados	Agregar pruebas instrumentadas y pruebas unitarias de IA.

Como trabajo futuro se propone agregar selección de dificultad, persistencia del estado, accesibilidad (descripciones de botones y contraste), pruebas unitarias parametrizadas para todos los tableros y una biblioteca compartida para reducir duplicación entre las implementaciones.

Conclusiones

Se completó la organización solicitada de seis carpetas Android: Java y Kotlin para MediumAI, UnbeatableAI y UnbeatableAI-MinMax. La carpeta 4 fue migrada a partir del proyecto Kotlin existente y la carpeta 3 fue incorporada como variante Kotlin de IA óptima. Todas las aplicaciones conservan el flujo principal del juego, el reinicio y la detección de resultados.

La comparación muestra que MediumAI es apropiada para una dificultad intermedia por su bajo costo y comportamiento variable, mientras que las variantes óptimas son adecuadas cuando se exige que la IA no pierda. La separación entre actividad y algoritmo permite validar las seis alternativas con los mismos criterios funcionales y facilita futuras iteraciones bajo SRUM+XP.

Referencias técnicas

- IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.
- Android Developers, *Build your first Android app* y documentación de Android SDK.
- Beck, K., *Extreme Programming Explained: Embrace Change*.
- Código fuente y README de los seis proyectos incluidos en este repositorio.